Analyzing E-Learning Platform Purchases Using MYSQL

Project Objective:The main objective of this project is to extend a relational database to derive meaningful insights from the e-learning purchase data.

Dataset Description:The given Dataset contains three tables such as Learners,Courses and Purchases with given attributes and constraints.

Column Description:

Table:Learners

Column Name	Data Type
Learner_id	It should be unique so it is Primary Key
Full_name	It contains name of the learners so it is VARCHAR
Country	It contains country of residence so it is VARCHAR

Table:Courses

Column Name	Description
Course_id	It should be unique so it is Primary Key
Course_name	It contains name of the courses so it is VARCHAR
Category	It contains category of the courses

	so it is VARCHAR
Unit_price	It is in Decimal data type

Table:Purchases

Column name	description
Purchase_id	It is the Primary key because it should be unique.
Learner_id	It is reference from learner table so it is Foreign Key
Course_id	It is reference from course table so it is Foreign Key
Quantity	It is in integer data type
Purchase_Date	It is in DATE data type

Tasks:
a)Database Setup&Data Entry:

1.Create a new database:
Command Used:CREATE DATABASE IF NOT EXISTS purchasedb;
USE purchasedb;
A database named purchasedb is created and using that database data is stored.

```
CREATE DATABASE IF NOT EXISTS purchasedb;
USE purchasedb;
```

2.Create Three tables with proper data types:

Command Used:CREATE TABLE IF NOT EXISTS learners()
CREATE TABLE IF NOT EXISTS courses()
CREATE TABLE IF NOT EXISTS purchases()

Tables are created with specific data types.Unique values are given as primary key and some attributes are given as Foreign Key.

Learners Table:

```
CREATE TABLE IF NOT EXISTS learners(
Learner_id INT PRIMARY KEY,
Full_name VARCHAR(100),
Country VARCHAR(100)
);
```

Courses Table:

```
CREATE TABLE IF NOT EXISTS courses(
Course_id INT PRIMARY KEY,
Course_name VARCHAR(100),
Category VARCHAR(100),
Unit_price DECIMAL(10,2)
);
```

Purchases Table:

```
CREATE TABLE IF NOT EXISTS purchases(
Purchase_id INT PRIMARY KEY,
Learner_id INT,
Course_id INT,
FOREIGN KEY(Learner_id) REFERENCES learners(Learner_id),
FOREIGN KEY(Course_id) REFERENCES courses(Course_id),
Quantity INT,
Purchase_Date DATE
);
```

3.Inserting Sample data:

Command Used:INSERT INTO learners(Learner id,Full name,Country) VALUES(101,'John','Canada'),

(102,'Sophia','Germany'),

(103,'David','France'),

(104,'James','Italy'),

(105,'Olivia','Japan'),

(106,'Sneha','India');

```
SELECT*FROM learners;

INSERT INTO courses(Course_id,Course_name,Category,Unit_price)
VALUES(301,' Business Intelligence','Business',5000),
(302,'Machine Learning','Data',10000),
(303,'Web development','Programming',8000),
(304,'Cloud Computing','Cloud',15000),
(305,'Deep Learning','Artificial Intelligence',20000),
(306,'SQL','Database',25000);
SELECT*FROM courses;

INSERT INTO
purchases(Purchase_id,Learner_id,Course_id,Quantity,Purchase_Date)
VALUES(2001,101,301,2,'2025-09-28'),
(2002,102,302,Null,'2026-02-14'),
(2003,103,303,1,'2025-08-27'),
(2004,104,304,3,'2026-01-19'),
(2005,105,305,2,'2026-05-31'),
(2006,102,303,3,'2025-04-18'),
(2007,106,306,4,'2026-03-22'),
(2008,104,302,5,'2025-06-25');
SELECT*FROM purchases;
```

b)Data Exploration using Joins:

1.Combine learners,Course,Purchase Data:

Command Used:SELECT

```
l.Learner_id,l.Full_name,c.Course_name,c.Category,p.Quantity,p.Purchase
_date
FROM Learners AS l
INNER JOIN purchases as p
ON l.Learner_id=p.Learner_id
INNER JOIN courses AS c
ON p.Course_id=c.Course_id;

ALTER TABLE purchases
ADD Total_Amount DECIMAL(10,2);

UPDATE purchases AS p
INNER JOIN courses AS c
ON p.Course_id=c.Course_id
SET p.Total_Amount=COALESCE(p.Quantity,0)*c.Unit_Price;
SELECT*FROM purchases;

SELECT
l.Full_name AS Learner_name,
c.Course_name AS Course_name,
c.Category AS Course_category,
p.Quantity AS Quantity,
FORMAT(p.Total_Amount,2) AS Total_Amount,
p.Purchase_Date
```

FROM purchases AS p

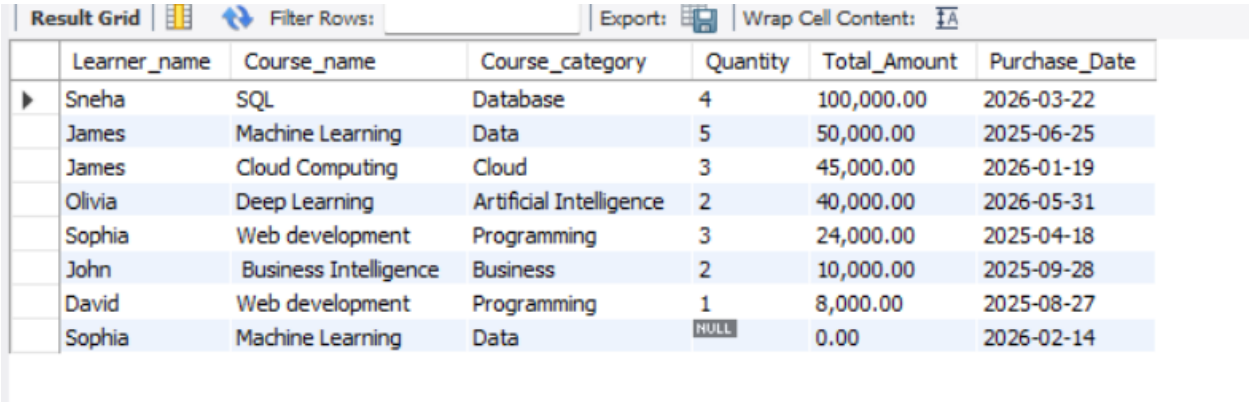
INNER JOIN learners AS l

ON p.Learner_id=l.Learner_id

INNER JOIN courses AS c

ON p.Course_id=c.Course_id

ORDER BY p.Total_Amount DESC;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner_name	Course_name	Course_category	Quantity	Total_Amount	Purchase_Date
Sneha	SQL	Database	4	100,000.00	2026-03-22
James	Machine Learning	Data	5	50,000.00	2025-06-25
James	Cloud Computing	Cloud	3	45,000.00	2026-01-19
Olivia	Deep Learning	Artificial Intelligence	2	40,000.00	2026-05-31
Sophia	Web development	Programming	3	24,000.00	2025-04-18
John	Business Intelligence	Business	2	10,000.00	2025-09-28
David	Web development	Programming	1	8,000.00	2025-08-27
Sophia	Machine Learning	Data	NULL	0.00	2026-02-14

Here,Total Amount is calculated using Quantity*Unit Price and they are sorted by highest total amount using DESC command.

Currency is formatted to 2 decimal places using FORMAT command.

For course name,category alias name are given as course_name,course_category.Three tables are combined using INNER JOIN and output is displayed.

c)Core Analytical Queries:

1.Display each learner's total spending with their country:

Command Used:SELECT

l.Full_name AS Learner_name,

l.Country,

SUM(p.Total_Amount) AS Total_Spending

FROM learners AS l

INNER JOIN purchases AS p

ON l.Learner_id=p.Learner_id

GROUP BY

l.Learner_id,

l.Full_name,

l.Country;

Total spending is calculated using Sum() function and using INNER JOIN learners total spending with respective country is calculated using GROUP BY.

Learner_name	Country	Total_Spending
John	Canada	10000.00
Sophia	Germany	24000.00
David	France	8000.00
James	Italy	95000.00
Olivia	Japan	40000.00
Sneha	India	100000.00

2.Top 3 most purchased courses by quantity:

Command Used:SELECT

c.Course_Name AS Course_Name,

SUM(p.Quantity) AS Total_Quantity

FROM purchases AS p

INNER JOIN Courses AS C

ON p.Course_id=c.Course_id

GROUP BY

c.Course_id,

c.Course_Name

ORDER BY Total_Quantity DESC

Limit 3;

Top 3 purchased courses are calculated using INNER JOINand GROUP BY. With ORDER BY and Limit 3, the first three purchased courses are calculated.

Course_Name	Total_Quantity
Machine Learning	5
SQL	4
Web development	4

3.Show each category's Total revenue and number of unique Learners:

Command used:SELECT

c.Category AS Category,

SUM(p.Total_Amount) AS Total_Revenue,

COUNT(DISTINCT p.Learner_id) AS Unique_Learners

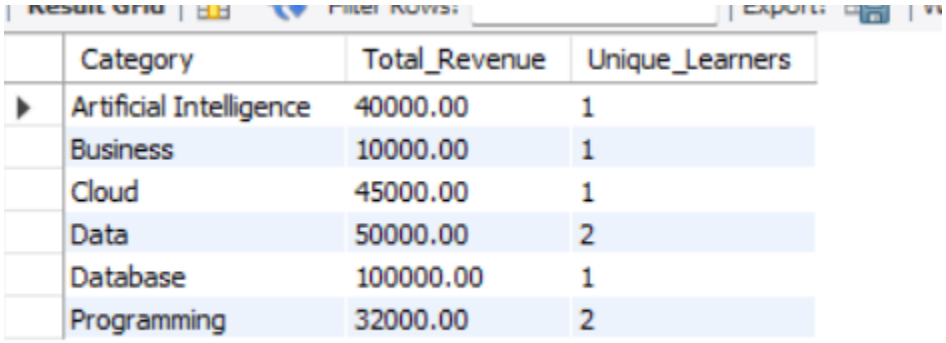
FROM purchases AS p

INNER JOIN courses AS c

ON p.Course_id=c.Course_id

GROUP BY c.Category;

Categorywise total revenue is shown and unique learners are displayed using the DISTINCT function.



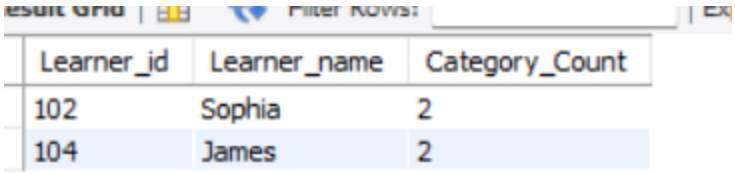
Category	Total_Revenue	Unique_Learners
Artificial Intelligence	40000.00	1
Business	10000.00	1
Cloud	45000.00	1
Data	50000.00	2
Database	100000.00	1
Programming	32000.00	2

4.Learners who purchased more than one category:

Command used:SELECT

```
l.Learner_id,
l.Full_name AS Learner_name,
COUNT(DISTINCT c.Category) AS Category_Count
FROM learners AS l
INNER JOIN purchases AS p
ON l.Learner_id=p.Learner_id
INNER JOIN courses AS c
ON p.COurse_id=c.Course_id
GROUP BY
l.Learner_id,
l.Full_name
HAVING COUNT(DISTINCT c.Category)>1;
```

Here learners count is calculated using HAVING,COUNT and DISTINCT functions.



Learner_id	Learner_name	Category_Count
102	Sophia	2
104	James	2

5.Identify courses never purchased:

Command Used:SELECT

c.Course_id,

c.Course_Name,

c.Category

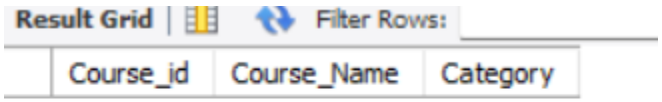
FROM courses AS c

LEFT JOIN purchases AS p

ON c.Course_id=p.Course_id

WHERE p.Course_id IS NULL;

Here LEFT JOIN is used and IS NULL also used to check purchase of courses .



Result Grid | Filter Rows:

Course_id	Course_Name	Category

But in this database there is no null value for purchase of courses.

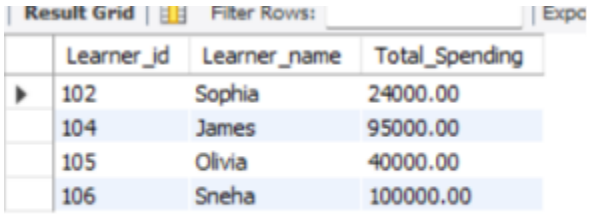
d)**CTE,Case,View and Null handling:**

1.Use CTE(Common TAble Expressions) to calculate total spending and to display learners with spending above 10,000.

Command Used: WITH learner_spending AS (

SELECT

```
l.Learner_id,
l.Full_name,
SUM(p.Total_amount) AS Total_spending
FROM learners AS l
INNER JOIN purchases AS p
ON l.Learner_id=p.Learner_id
GROUP BY
l.Learner_id,
l.Full_name
)
SELECT
Learner_id,
Full_name AS Learner_name,
Total_Spending
FROM learner_spending
WHERE Total_spending>10000;
```


Result Grid | Filter Rows: | Expo

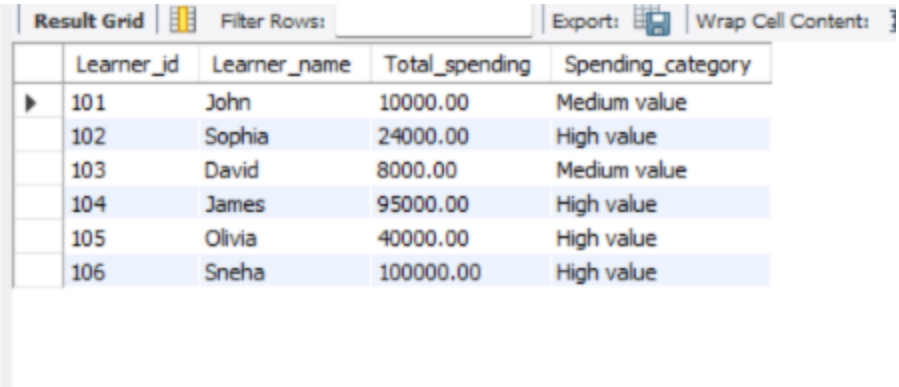
Learner_id	Learner_name	Total_Spending
102	Sophia	24000.00
104	James	95000.00
105	Olivia	40000.00
106	Sneha	100000.00

To display total spending per learner is calculated then total spending above 10000 is calculated using CTE.

2.Classify learners based on spending using CASE Expression:

Command Used: WITH learner_spending AS (

```
SELECT
l.Learner_id,
l.Full_name,
SUM(p.Total_amount) AS Total_spending
FROM learners AS l
INNER JOIN purchases AS p
ON l.Learner_id=p.Learner_id
GROUP BY
l.Learner_id,
l.Full_name
)
SELECT
Learner_id,
Full_name AS Learner_name,
Total_spending,
CASE
WHEN Total_spending>15000 THEN 'High value'
WHEN Total_spending>=8000 THEN 'Medium value'
ELSE 'Low value'
END AS Spending_category
FROM learner_spending;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner_id	Learner_name	Total_spending	Spending_category
101	John	10000.00	Medium value
102	Sophia	24000.00	High value
103	David	8000.00	Medium value
104	James	95000.00	High value
105	Olivia	40000.00	High value
106	Sneha	100000.00	High value

Using CASE Expression based on spending,learners is classified as

- Above 15,000-”High Value”
- 8,000-15,000-”Medium Value”
- Below 8,000-”Low Value”

3.Display all courses and replace NULL purchase counts with COALESCE():

Command Used: SELECT

```
c.Course_id,
c.Course_name,
COALESCE(COUNT(p.Purchase_id),0) AS Purchase_Count
FROM courses AS c
LEFT JOIN purchases AS p
ON c.Course_id=p.Course_id
GROUP BY
c.Course_id,
c.Course_name;
```

Course_id	Course_name	Purchase_Count
301	Business Intelligence	1
302	Machine Learning	2
303	Web development	2
304	Cloud Computing	1
305	Deep Learning	1
306	SQL	1

Since the table has no null value it returns only purchase count.

4.Create Category_Performance_view showing Category,Total revenue,Number of purchases,Average revenue per purchases.

Command Used: CREATE VIEW Category_performance_view AS

SELECT

c.category,

SUM(p.Total_amount) AS Total_revenue,

COUNT(*) AS Number_of_purchases,

AVG(p.Total_amount) AS Average_revenue_per_purchase

FROM courses AS c

INNER JOIN purchases AS p

ON c.course_id=p.course_id

GROUP BY c.category;

SELECT*FROM Category_performance_view;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	Total_revenue	Number_of_purchases	Average_revenue_per_purchase
Business	10000.00	1	10000.000000
Data	50000.00	2	25000.000000
Programming	32000.00	2	16000.000000
Cloud	45000.00	1	45000.000000
Artificial Intelligence	40000.00	1	40000.000000
Database	100000.00	1	100000.000000

Conclusion:

This e-learning purchase analysis project demonstrates how MySQL can be used to transform raw purchase data into meaningful business insights. By analyzing learners, courses, categories, and purchases, the project helps identify **sales trends, learner purchasing behaviour, and popular course categories**.

The use of **joins, aggregate functions, subqueries, CTEs, CASE statements, views, and window functions** enables to understand revenue patterns, measure course and category performance, and support data-driven decision-making.